

Technical Product Requirements Document

Legal Metrology Packaged Commodities Compliance System — SIH26034

Engineering Specification — v1.0

1. Problem Statement

PS ID: SIH26034 | **Ministry:** Consumer Affairs, Food & Public Distribution | **Dept:** DoCA

Every packaged commodity sold in India must carry mandatory declarations under the Legal Metrology Act 2009 and the Legal Metrology (Packaged Commodities) Rules 2011 — manufacturer details, net quantity, MRP, manufacture date, consumer care info, and more, in a legally specified format (font size, placement, wording). Enforcement today is manual: inspectors physically check labels against a 40+ page rulebook, one product at a time. This does not scale — e.g., in Feb 2026, a single CCPA review of e-commerce listings found 16,970 non-compliant listings in one product category alone.

Objective: Build a system that takes a product-label image (from a camera or an e-commerce listing screenshot) and automatically determines Legal Metrology compliance, replacing manual rule-by-rule checking with an automated pipeline usable by consumers, companies, and enforcement officials alike.

2. Users & Roles

Role	Who	Primary need
user	General public/consumer	Scan any product, file a complaint if non-compliant
company	Business self-checking own products	Pre-launch/ongoing compliance verification
inspector	Field/enforcement official	Official case creation, cross-company audit history
admin	Consumer Affairs oversight	Full analytics, complaint triage, company risk ranking

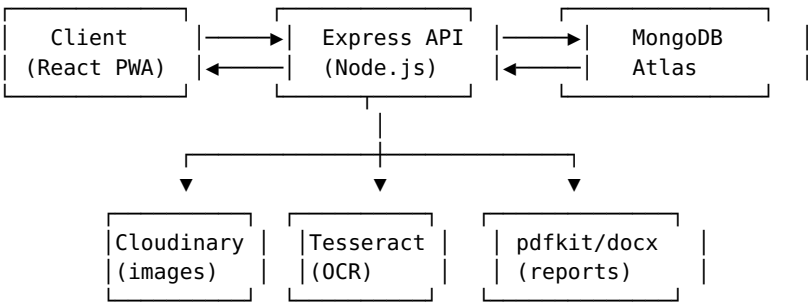
3. Legal Rule Set Implemented (verified against official Gazette text)

Rule	Requirement	Engine implementation
6(a)	Manufacturer/packer/importer name+address	Regex + address-block detection. Skipped for food (FSSAI governs instead)
6(aa)	Country of origin	Required only if category == electronics or import flag set
6(b)	Generic/common name	Keyword match against category taxonomy
6(c)	Net quantity + unit	Regex: \d+\s?(g\ kg\ ml\ l\ gm)
6(d)	Mfg month/year	Skipped for food (FSSAI) and cosmetics (Drugs & Cosmetics Rules)
6(e)	MRP “inclusive of all taxes”	Regex for ₹ + amount + required phrase
6(2)	Consumer care (phone/email)	Regex: phone pattern OR email pattern
6(7)	GM label (food only)	Text search “GM” near top of image — conditional on category
6(8)	Veg/non-veg dot	Flagged as “requires visual verification” — color-detection, not OCR; do not fake-detect
7	Font size vs PDP area	See §5.3 — area-based table, not weight-based (corrected from pre-2018 rule)
8	Placement on PDP	Bounding-box clustering heuristic — see §5.4
9	Readability/contrast	OCR confidence score used as readability proxy Regex block-list: “minimum”, “not

12(6)	No misleading words	less than", "average", "about", "approximately" ≤10g/10ml (except tobacco), restaurant fast-food, drugs,
26	Exemptions	cosmetics — engine must check category+quantity before applying any rule

Design principle: All of the above rules are stored as **data in the rules collection**, not hardcoded in the rule-engine function. The engine reads active rules at scan-time and applies them generically. This means a future amendment (e.g. the April 2026 e-commerce country-of-origin amendment) is handled by inserting a new document, not by redeploying code.

4. System Architecture



Request flow (a scan): 1. Client uploads image → POST /api/scans 2. Image streamed to Cloudinary, URL returned 3. Tesseract.js runs OCR on the image → raw text + bounding boxes 4. Extraction service applies regex rules (fetched from DB) → structured fields 5. Rule engine validates fields against active rules for the product’s category 6. Result persisted to scans + violations collections 7. If violations exist and companyId is set, riskScore updated 8. Response returned to client with compliance result

5. Backend Design

5.1 Folder Structure

```

/backend
  /config      - db.js, cloudinary.js, index.js (env-driven config)
  /models      - *.model.js (Mongoose schemas)
  /controllers - *.controller.js (request handlers, thin – delegate
to services)
  /services    - *.service.js (business logic: ocr.service.js,
```

```

ruleEngine.service.js,
    riskScore.service.js, report.service.js)
  /routes      - *.route.js (Express routers, one per resource)
  /middleware  - auth.middleware.js, authorize.middleware.js,
error.middleware.js,
    upload.middleware.js (multer config)
  /validators  - *.validator.js (input validation, e.g.
Joi/express-validator schemas)
  /utils       - *.util.js (regex patterns, PDP-area calculator,
response formatter)
  app.js       - Express app setup, middleware mounting
  server.js    - entrypoint, DB connection, listen

```

Layering principle: Controllers never contain business logic directly — they parse the request, call a service, and format the response. Services never touch req/res. This separation is what makes the rule-engine testable independently of HTTP.

5.2 Authentication & Authorization

- POST /api/auth/signup, POST /api/auth/login issue JWT (payload: {userId, role, companyId}), signed with JWT_SECRET, expiry ~7 days.
- auth.middleware.js — verifies JWT, attaches req.user.
- authorize.middleware.js — factory function authorize(['admin', 'inspector']) that checks req.user.role against an allowed list; returns 403 if not permitted.
- Passwords hashed with bcrypt (cost factor 10+), never stored/logged in plaintext.

5.3 Font-size validation logic

1. Get bounding-box pixel height of the net-quantity numeral (from OCR)
2. Get package dimensions:
 - If user provided width×height (cm) → use directly
 - Else → default to a 100cm² PDP area assumption, flag result as "estimated"
3. Calculate PDP area:
 - Rectangular: height × width
 - Cylindrical: 0.4 × (height × circumference)
 - Unknown shape: default to rectangular formula
4. Look up minimum required mm height from the CURRENT (post-2018) table:

≤50cm²→1.0mm	50-100→1.5mm	100-500→2.5mm	500-2500→4.0mm	>2500→6.0mm
--------------	--------------	---------------	----------------	-------------
5. Convert detected pixel height to mm using the pixel-to-cm calibration
 - ratio derived from provided/assumed package dimensions
6. Compare against minimum — pass/fail, always labeled "estimated" unless
 - real dimensions were provided

5.4 Placement/clustering heuristic

1. Collect bounding-box center-points of all detected mandatory fields

2. Compute average position (centroid) of all fields
3. For each field, compute distance from centroid as % of image height
4. If any field's distance > 40% threshold → flag "placement may not comply with Principal Display Panel grouping (Rule 8)"
(heuristic red-flag, not a definitive legal determination – stated as such in the output)

5.5 API Route Map

Method	Route	Role	Purpose
POST	/api/auth/signup	public	Register user
POST	/api/auth/login	public	Login, returns JWT
POST	/api/scans	user, company, inspector	Upload image, run full pipeline
GET	/api/scans	all (filtered)	List own scans (paginated) — admin sees all
GET	/api/scans/:id	owner, inspector, admin	Single scan detail
GET	/api/scans/:id/report	owner, inspector, admin	Generate/download PDF or DOCX
POST	/api/complaints	user	File complaint from a non-compliant scan
GET	/api/complaints	admin, inspector (assigned)	List complaints (paginated)
PATCH	/api/complaints/:id	admin	Assign to inspector / change status
POST	/api/cases	inspector	Create official case from a scan
PATCH	/api/cases/:id	inspector (own, if not locked)	Update draft case
POST	/api/cases/:id/lock	inspector	Lock case (immutability trigger)
POST	/api/cases/:id/addendum	inspector	Add addendum to locked case
GET	/api/companies	admin, inspector	List companies, sorted by riskScore
GET	/api/companies/:id/history	inspector, admin	Full audit history for a company
GET	/api/rules	admin	List/manage active rules
POST	/api/rules	admin	Add new rule (for handling amendments)

GET	/api/dashboard/summary	admin	Aggregate stats fo dashboard
-----	------------------------	-------	------------------------------

Pagination convention (all list routes): ?page=1&limit=20, response includes { data, totalCount, page, totalPages }. No route returns an unbounded array.

Error convention: all errors return { success: false, message, code } via a centralized error.middleware.js — controllers throw, middleware catches.

5.6 Database Schema (see DB_DESIGN.md for full field-by-field detail)

9 collections: users, companies, products, rules, scans, violations, complaints, cases, reports. Key points:

- rules is DB-driven, versioned via effectiveFrom/effectiveTo.
- companies.riskScore (indexed) drives the inspector “priority list” feature via Company.find().sort({riskScore:-1}).
- violations.companyId and scans.scannerRole are intentionally denormalized for dashboard aggregation performance.
- cases.status = 'locked' blocks direct mutation at the route layer; only parentCaseId-linked addendum documents are permitted after lock.

6. Frontend Design

6.1 Tech Stack

React + Tailwind CSS, mobile-first responsive layout (PWA-capable — no native app build needed). State management: React Context + hooks is sufficient at this scale (no Redux needed). API calls via a thin apiClient.js wrapper around fetch/axios with interceptor for attaching JWT and handling 401s (auto-redirect to login).

6.2 Route Map (React Router)

Path	Component	Access
/login, /signup	Auth pages	public
/scan	Upload/camera capture page	user, company, inspector
/scan/:id/result	Result page — pass/fail per field	owner
/scan/:id/report	Report view + PDF/DOCX download	owner, inspector, admin
/history	Own scan history (paginated table)	user, company, inspector

/complaints/new/:scanId	File a complaint	user
/complaints	Complaint list/triage	admin, inspector
/companies	Company list sorted by risk	inspector, admin
/companies/:id/history	Full audit history for a company	inspector, admin
/cases/:id	Case detail, lock/addendum actions	inspector
/dashboard	Analytics dashboard (charts, stat cards)	admin
/admin/rules	Rule management (add/deactivate)	admin

6.3 Component Structure (high level)

```

/src
  /components
    /common      - Button, Card, Badge, Modal, Pagination,
LoadingSpinner
    /scan        - ImageUploader, CameraCapture, ScanResultCard,
FieldStatusRow
    /dashboard   - StatCard, ViolationBarChart, TrendLineChart,
CompanyRiskTable
    /complaints  - ComplaintForm, ComplaintCard,
ComplaintStatusBadge
    /pages       - one file per route above, composes components
    /context     - AuthContext (user, token, role)
    /hooks       - useAuth, usePaginatedFetch, useScanUpload
    /services    - apiClient.js, scanService.js,
complaintService.js, etc.
    /utils      - formatters, roleGuards (route-level role check)

```

Role-gated rendering: a `<RequireRole roles={['admin']}>` wrapper component checks `AuthContext` and either renders children or redirects — applied both at the route level (React Router `loader/guard`) and for conditionally showing sidebar links, so a company user never even sees a “Dashboard” link in their nav.

6.4 Key UX states to handle explicitly

- Upload in progress (loading skeleton, not a blank screen)
- OCR/low-confidence result (“Low confidence — retake photo” rather than a silent wrong answer)
- Empty states (no scans yet, no complaints yet) — not a blank table
- Network/API error (toast + retry option, never a raw stack trace)

7. Non-Functional Requirements

Concern	Approach
Scalability	Indexed queries, pagination everywhere, DB-driven rules (no

	redeploy for legal changes)
Security	JWT auth, bcrypt hashing, role-based route guards on both frontend and backend, input validation on every POST/PATCH
Data integrity	Locked-case immutability, addendum-only edits post-lock
Cost	100% free-tier stack (MongoDB Atlas, Cloudinary, Tesseract.js, Render/Railway, Vercel)
Honesty in output	Estimated values (font-size without real dimensions) explicitly labeled as estimates, never presented as precise measurements

8. Explicitly Out of Scope

- Automated web-crawling of e-commerce sites (legally risky, not requested by the PS)
 - Trained ML/computer-vision models (OCR is pre-trained and sufficient; rule logic is deterministic, not statistical)
 - Laboratory-precise font measurement (physically impossible from a photo without a calibration reference)
 - Real-time government-system API integration (no such API is available to a student team — the complaints collection is the realistic proxy for this)
-

9. Build Sequence (checkpoint order)

1. Project scaffold + DB connection + health check
2. Mongoose models (all 9 collections)
3. Auth (signup/login/JWT/role middleware)
4. Image upload pipeline (Cloudinary)
5. OCR integration (Tesseract.js) + bounding-box extraction
6. Rule engine (DB-driven, regex-based field extraction + validation)
7. Evidence/case model + locking logic
8. History/repository endpoints (paginated)
9. PDF/DOCX report generation
10. Dashboard aggregation endpoints
11. Frontend — only once wireframes are available, in the same order: auth pages → upload/result → history → complaints → dashboard